

Object Model reactively with the help of DOM.

- `<p v-if="seen">Now you see me</p>`

In this example, based on the truthfulness of the `seen` expression value, the `v-if` directive can change the `<div>` element. It can either remove the element or get it inserted.

Arguments

Some directives may take the form of an argument in Vue. You can use it after the directive name is denoted by a colon. Here is an example that depicts the `v-bind` directive:

- `<a v-bind:href="url"> ... </a>`

In this example, `v-bind` is the directive and `href` is the argument. `Href` directs the directive to bind the `href` attributes to the expression value of the url.

Let's look at another example with a `v-on` directive. This follows the DOM events:

- `<a v-on:click="doSomething"> ... </a>`

Here, the value of the `eventName` is "`focus`," there will be an equivalency between `v-on:[eventName]` and `v-on:focus`.

Dynamic Argument Value Constraints

The one that evaluates to a string, dynamic arguments puts forth the null exception. You can explicitly remove the binding with the special value `null`. It will throw a warning if there is any other non-string value.

Dynamic Argument Expression Constraints

There are some limitations with respect to the syntax for dynamic argument expressions. A few characters, such as quotes and spaces, do not stand valid while using the HTML attribute names. For example, the following can be considered as null and void:

- `<!-- This will trigger a compiler warning. -->`
- `<a v-bind:['foo' + bar]="value"> ... </a>`

You can either replace the complex expression or use it without quotes or spaces with a computed property.

When you are picking the templates which are directly written in the HTML file like the in-DOM templates, it is best to prevent using uppercase characters, including naming keys as browsers will convert these attribute names into lowercase: